

## Week 1 Day 4 — Model Tuning, Regularization & Reproducible Pipelines

### 1. Objective

The objective of Day 4 was to turn the best candidate models from the previous experiments into robust, reproducible machine-learning pipelines. The Adult Census Income dataset was used to predict whether an individual earns more than \$50K per year. The primary optimization metric was **F1 score**, since the positive class is relatively less frequent and both precision and recall are important for identifying high-income individuals.

A stratified 80/20 hold-out split was maintained throughout the experiments, with the final test set kept untouched until final evaluation. All experiments used `random_state=42` for reproducibility. Preprocessing, feature transformation, and model fitting were combined inside sklearn pipelines to prevent data leakage. The documented environment was Python 3.14.3, NumPy 2.4.4, pandas 3.0.2, and scikit-learn 1.8.0. The dataset used was OpenML Adult version 2.

### 2. Reproducible Pipelines

The preprocessing pipeline handled numerical and categorical variables separately. Numerical features were median-imputed and standardized, while categorical features were imputed using the most frequent value and one-hot encoded with unknown categories ignored.

The main candidate models were Logistic Regression, Random Forest, and Gradient Boosting. Each estimator was placed after the preprocessing stage in a single sklearn Pipeline. This ensures that preprocessing is learned only from the training data during cross-validation and can be reproduced automatically when new observations are supplied.

The train-development portion contained **39,073 observations**, while **9,769 observations** were reserved as the untouched test set. The resulting pipelines were saved using joblib, together with model metrics and environment metadata.

### 3. Hyperparameter Search

Hyperparameters were tuned using **RandomizedSearchCV** with **5-fold StratifiedKFold cross-validation**. F1 was used as the primary optimization metric. The search explored regularization strength for Logistic Regression and model complexity parameters for the tree-based models.

The best configurations were:

| Model               | Best Hyperparameters                                                              |
|---------------------|-----------------------------------------------------------------------------------|
| Logistic Regression | <code>C=3, penalty=l1</code>                                                      |
| Random Forest       | <code>n_estimators=400, max_depth=20, min_samples_leaf=2, max_features=0.8</code> |
| Gradient Boosting   | <code>n_estimators=400, learning_rate=0.1, max_depth=4, min_samples_leaf=5</code> |

The search therefore identified relatively weak Logistic Regression regularization ( $C=3$ ) with L1 regularization, while the tree models benefited from controlled complexity through depth and minimum-leaf constraints.

Earlier cross-validation results showed Gradient Boosting as the strongest candidate, with mean F1 of **0.6811**, compared with **0.6728** for Logistic Regression. The difference was statistically significant but practically modest, so both models remained relevant candidates during tuning.

#### 4. Overfitting and Underfitting Diagnostics

Learning curves were used to examine whether additional training data substantially improved model performance. For the evaluated model, training F1 decreased from **0.7076** at the smallest training size to **0.6768** at the largest training size, while validation F1 increased from **0.6690** to **0.6729**.

The decreasing gap between training and validation performance suggests that severe overfitting was not the primary problem at larger training sizes. Instead, the curves indicate a relatively small remaining gap and limited improvement from additional data. This points toward a **moderate bias/underfitting limitation** rather than severe variance.

The practical implication is that simply adding more training observations may provide only modest improvement. Further gains are more likely to come from better feature engineering, appropriate regularization, and model/threshold optimization.

#### 5. Probability Calibration and Threshold Selection

The final candidate was a **sigmoid-calibrated Logistic Regression pipeline**. Calibration was important because the model's predicted probabilities are used to determine which individuals should be classified as likely high-income prospects.

Instead of automatically using the conventional 0.50 classification threshold, thresholds were evaluated on the development set. The threshold was selected specifically to maximize F1 while keeping the final test set untouched.

The selected threshold was **0.30**. This produced a better balance between precision and recall for the project's business objective. Using the default threshold of 0.50 resulted in an F1 score of **0.6736**, whereas the optimized threshold increased F1 to **0.7008** on the final test set. The lower threshold increased recall while reducing precision and overall accuracy, reflecting the trade-off between finding more high-income individuals and contacting more false-positive prospects.

#### 6. Final Test Performance

The final calibrated Logistic Regression pipeline was evaluated once on the untouched hold-out test set at the selected threshold of 0.30.

| Metric            | Final Score   |
|-------------------|---------------|
| Accuracy          | <b>0.8367</b> |
| Precision         | <b>0.6241</b> |
| Recall            | <b>0.7990</b> |
| F1 Score          | <b>0.7008</b> |
| ROC AUC           | <b>0.9135</b> |
| Average Precision | <b>0.7869</b> |
| Brier Score       | <b>0.0977</b> |

The confusion matrix contained **6,306 true negatives, 1,868 true positives, 1,125 false positives, and 470 false negatives**.

The final model therefore provides strong ranking ability, demonstrated by its **0.9135 ROC AUC**, while the optimized threshold provides substantially better recall than the default 0.50 threshold. The F1 score of **0.7008** represents the best final balance between precision and recall achieved in the completed tuning workflow.

## 7. Final Model and Production Behavior

The final artifact is a **sigmoid-calibrated Logistic Regression pipeline** using L1 regularization with C=3. Logistic Regression was selected for the final artifact because it provides competitive validation performance, interpretable probabilities, and a relatively compact production model. The preprocessing and calibration components remain attached to the saved pipeline.

The complete model was saved as `final_adult_income_pipeline.joblib` and verified after reloading. In production, incoming observations should undergo the same feature engineering and preprocessing used during training. The calibrated model produces the probability of earning more than \$50K, and the saved threshold of 0.30 converts that probability into the final classification.

Production performance should be monitored over time, particularly class balance, precision, recall, F1, and probability calibration, because changes in the underlying population or input data distribution may reduce model performance.